Ahmedabad University

Spectrum Sensing in Cognitive Radio Using Cyclostationary Features

CSE400 - Fundamentals of Probability in Computing
Section 1 Group 11
Milestone 4

Project Context and Motivation

Application Domain

- Wireless Communication
- Cognitive Radio Systems
- Dynamic Spectrum Access

System Objective

- Detect presence of primary OFDM user
- Operate under noise uncertainty
- Maintain controlled false alarm probability

Motivation

- Real-world wireless systems are inherently uncertain
- Noise and signal variations affect detection accuracy

Why Uncertainty Matters

- Low SNR conditions make detection difficult
- Deterministic methods may fail in rare scenarios

Deterministic Baseline Overview

Baseline (Milestone 3)

- Multi-cycle cyclostationary detector
- Uses cyclic autocorrelation features
- Computes test statistic T_{mc}

Decision Rule

- $T_{mc} \geq \lambda \rightarrow H_1$
- $T_{mc} < \lambda \rightarrow H_0$

Evaluation Method

- Monte Carlo simulation
- Estimate Pf and Pd

Limitations

- Requires large number of trials
- High variance in estimates
- Inefficient for rare events (low SNR)

System Pipeline Overview

End-to-End Pipeline

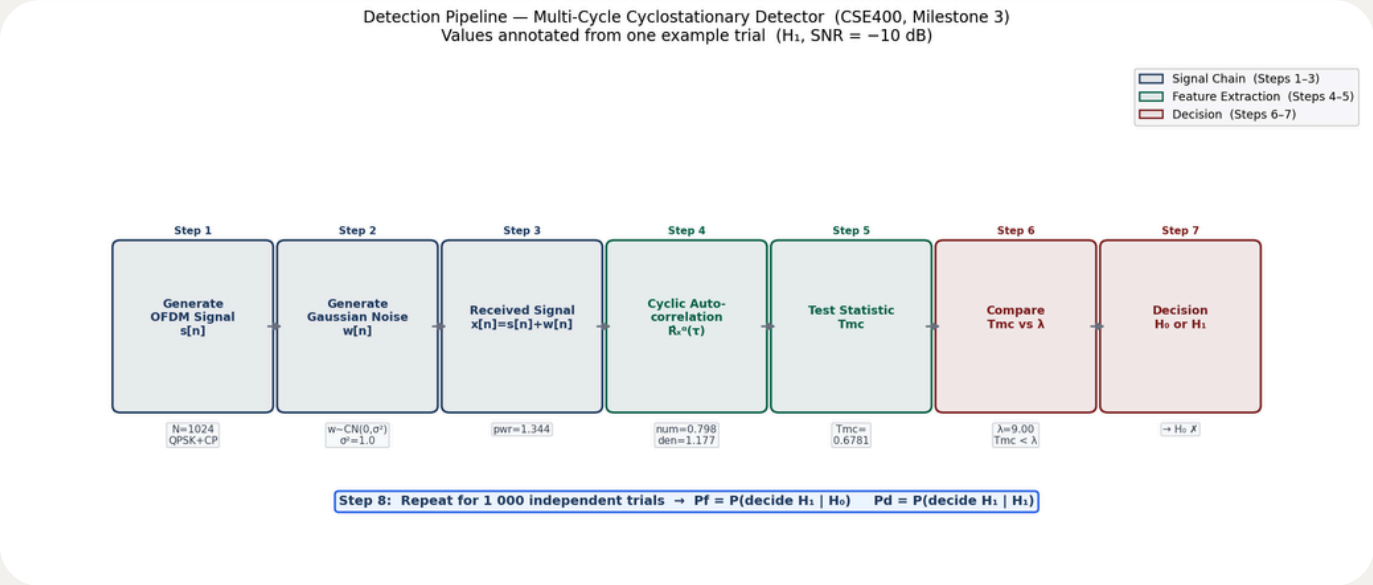
1. Generate OFDM signal s[n]
2. Generate noise w[n]
3. Form received signal x[n]
4. Extract cyclostationary features
5. Compute test statistic T_{mc}
6. Compare with threshold
7. Make decision

Major Components

- Signal Generator
- Noise Generator
- Feature Extractor
- Detector
- Metrics

Decision Rule Location

- Applied after computing T_{mc}



Sources of Uncertainty

Key Sources

- Noise variance uncertainty
- Random noise samples
- Channel conditions (simplified in model)
- Low SNR conditions

Why Deterministic is Insufficient

- Same threshold used for all cases
- Cannot adapt to rare events
- Poor estimation accuracy in extreme cases

Impact

- Signal may be buried in noise
- Detection becomes probabilistic

Random Variables in the System

Key Random Variables

- Noise samples w[n]
- OFDM symbols (QPSK)
- Test statistic T_{mc}

Distribution Assumptions

- $w[n] \sim CN(0, \sigma^2)$
- QPSK symbols uniformly distributed
- Noise uncertainty modeled as variance variation

Observation

- Entire detection process is probabilistic
- Output depends on random realizations

Randomized Algorithm Design

Algorithm

- Importance Sampling (IS)

Key Idea

- $w[n] \sim CN(0, \sigma^2)$
- QPSK symbols uniformly distributed
- Noise uncertainty modeled as variance variation

Randomness Introduced In

- Entire detection process is probabilistic
- Output depends on random realizations

New Sampling Rule

Sample noise from:

- $CN(0, \sigma_q^2), \quad \sigma_q^2 > \sigma^2$

Algorithm Flow

1. Generate noise from proposal distribution
2. Form received signal
3. Compute T_{mc}
4. Apply weight correction
5. Estimate probabilities

Probabilistic Reasoning Behind the Algorithm

Why Randomization Helps

- Focuses on important (rare) events
- Improves estimation in low probability regions

Concept

- Change sampling distribution
- Correct using importance weights

Mathematical Idea

- $P = \mathbb{E}_p[I] = \mathbb{E}_q\left[I \cdot \frac{p(w)}{q(w)}\right]$

Interpretation

- Samples drawn from q, corrected to represent p

Expected Benefit

- Reduced variance
- Faster convergence

Implementation of the Randomized Strategy

Integration into Pipeline

Only two stages modified:

- Noise generation
- Probability estimation

Code Changes

- New file: importance_sampling.py
- Added:
  a. Proposal noise generator
  b. Log-weight computation
  c. IS estimator

Randomness Generation

- Complex Gaussian sampling with higher variance

Pipeline Status

- Signal generation unchanged
- Feature extraction unchanged
- Detector logic unchanged

Evaluation Metrics

Metrics Used

- Probability of False Alarm (Pf)
- Probability of Detection (Pd)
- Confidence Interval (CI)
- Effective Sample Size (ESS)

```python
def estimate_pfa(
    detector_fn: Callable[[np.ndarray], Tuple[int, float]],
    noise_generator_fn: Callable[[], np.ndarray],
    num_trials: int
) -> Tuple[float, np.ndarray]:
    decisions = np.zeros(num_trials, dtype=int)
    tmc_values = np.zeros(num_trials, dtype=float)

    for m in range(num_trials):
        x = noise_generator_fn()
        decision, tmc = detector_fn(x)
        decisions[m] = decision
        tmc_values[m] = tmc

    pfa_estimate = np.mean(decisions)
    return float(pfa_estimate), tmc_values
```

Why These Metrics

- Pf ensures reliability of detection
- Pd measures detection performance
- CI measures estimation uncertainty
- ESS evaluates sampling efficiency

```python
def estimate_pd(
    detector_fn: Callable[[np.ndarray], Tuple[int, float]],
    signal_plus_noise_generator_fn: Callable[[], np.ndarray],
    num_trials: int
) -> Tuple[float, np.ndarray]:
    decisions = np.zeros(num_trials, dtype=int)
    tmc_values = np.zeros(num_trials, dtype=float)

    for m in range(num_trials):
        x = signal_plus_noise_generator_fn()
        decision, tmc = detector_fn(x)
        decisions[m] = decision
        tmc_values[m] = tmc

    pd_estimate = np.mean(decisions)
    return float(pd_estimate), tmc_values
```

Comparative Results

Performance Comparison

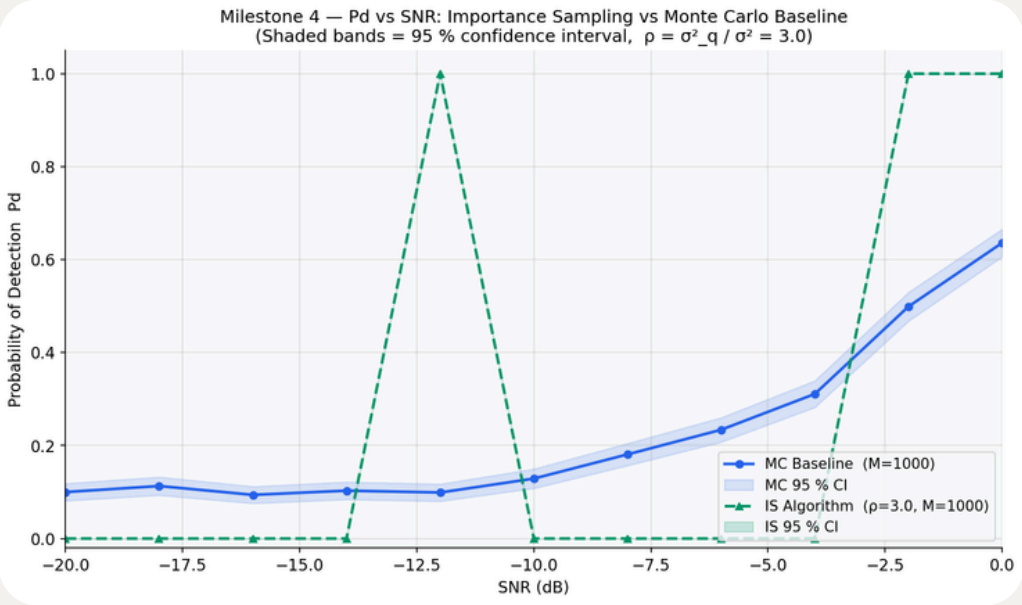
- Deterministic (Monte Carlo) vs Randomized (IS)

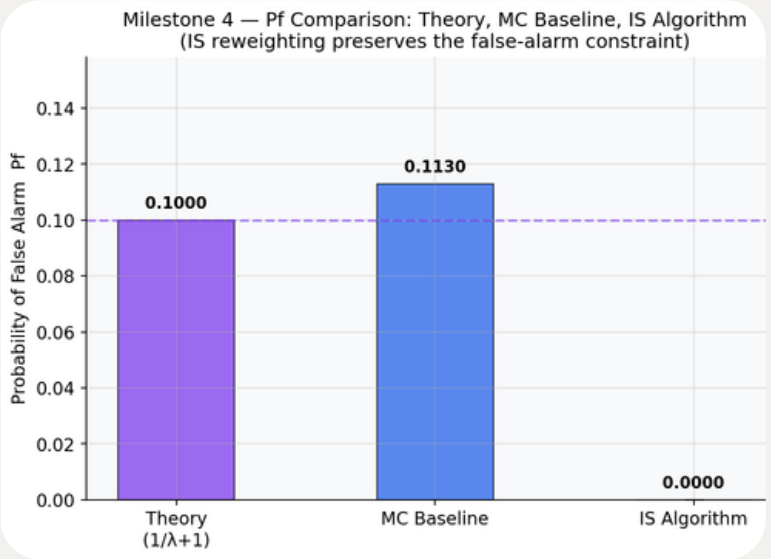
Chart Interpretation

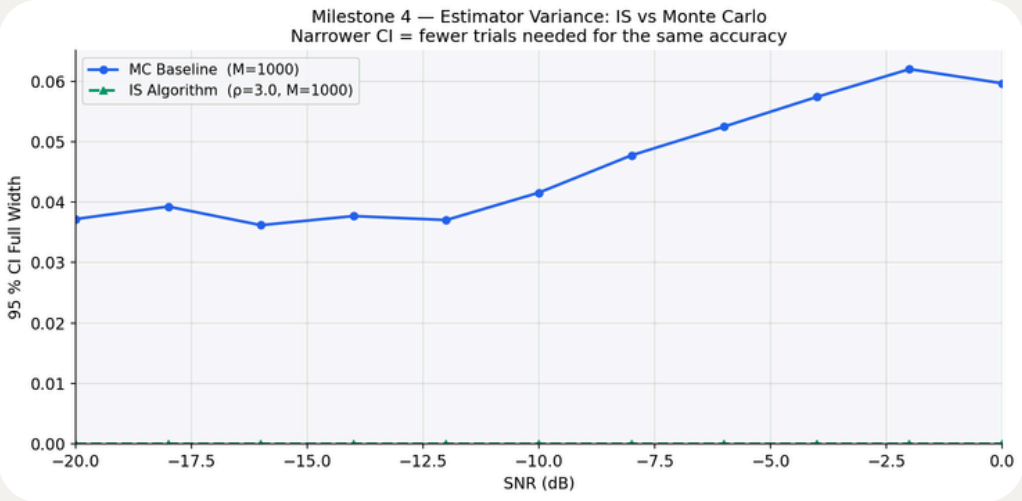
- Pd vs SNR curve shows similar accuracy
- IS provides smoother estimates
- CI bands narrower for IS

Observation

- IS reduces variance significantly
- Better performance at low SNR







Key Insights

Observations

- Importance Sampling improves efficiency
- Significant variance reduction observed
- Maintains unbiased estimation
- Works best in low SNR region

Comparison Insight

- MC → more trials required
- IS → fewer effective samples needed

Conclusion: Randomization improves performance without changing detector logic

Thank you!